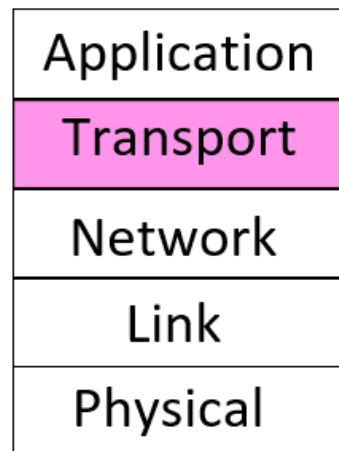


Capítulo 3

Capa de Transporte Direccionamiento



Direccionamiento

- **Problema:** El **direccionamiento** explícito de los destinos.
 - ¿Cómo hacer para que un proceso servidor *adecuado atienda* a las necesidades de una máquina cliente?
 - Consideraremos los siguientes casos:
 - El cliente podría **no saber cuál proceso servidor** es el adecuado para un servicio particular.
 - El cliente podría saber cuál proceso servidor es adecuado, pero dicho **proceso servidor no atiende porque esta inactivo**.

Direccionamiento

- **Problema:** El cliente necesita un tipo de servicio.
 - Sin embargo **no sabe cuál proceso servidor** es adecuado para contactar.
 - Asumir que los procesos servidores están activos.
 - **¿Cómo hacer que el cliente se entere de un proceso servidor para un tipo de servicio?**

Direccionamiento

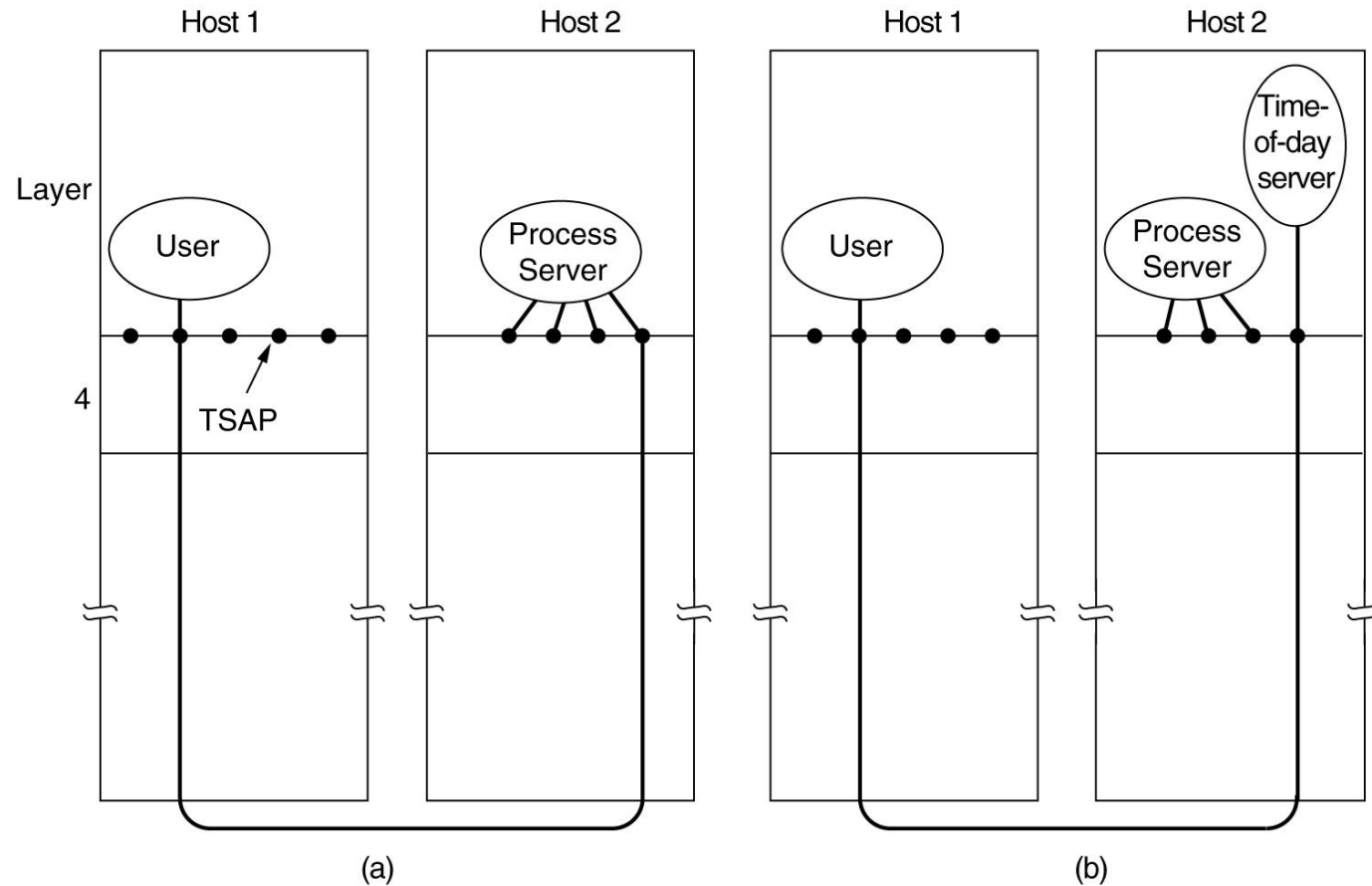
- **Solución:** Existe un proceso especial llamado **servidor de directorio** que para cada tipo de servicio sabe cuáles son los puertos de los servidores que prestan ese tipo de servicio.
 - Pasos seguidos:
 1. El usuario establece una conexión con el servidor de directorio (que escucha en un puerto bien conocido).
 2. El usuario envía un mensaje especificando el nombre del servicio.
 3. El servidor de directorio le devuelve la dirección puerto.
 4. El usuario libera la conexión con el servidor de directorio y establece una nueva con el servicio deseado.
- **¿Cómo se hace cuando se crea un servicio nuevo?**
 - El servicio nuevo debe registrarse en el servidor de directorio, dando su nombre de servicio como la dirección de su puerto.
 - El servidor de directorio registra esta información en su base de datos.

Caso 2: Servidor Inactivo → Protocolo Inicial de Conexión

- **Problema:** El cliente conoce al proceso servidor adecuado, pero este se encuentra **inactivo** y no puede responder a solicitudes.
 - El servidor existe en la máquina remota, pero su proceso no está corriendo en ese momento.
 - Se necesita un mecanismo para **activar** al servidor bajo demanda cuando llega una solicitud.
- **Solución:** Usar un **servidor de procesos (protocolo inicial de conexión)**, un intermediario que activa servidores inactivos cuando se los necesita.
 - El **servidor de procesos** escucha en los puertos de servidores de menor uso que están inactivos.
 - Cuando recibe una solicitud, **genera (fork)** el servidor correspondiente y le transfiere la conexión del cliente.
 - Los pasos detallados se muestran en las figuras de las siguientes diapositivas.

Protocolo Inicial de Conexión — Pasos 1 y 2

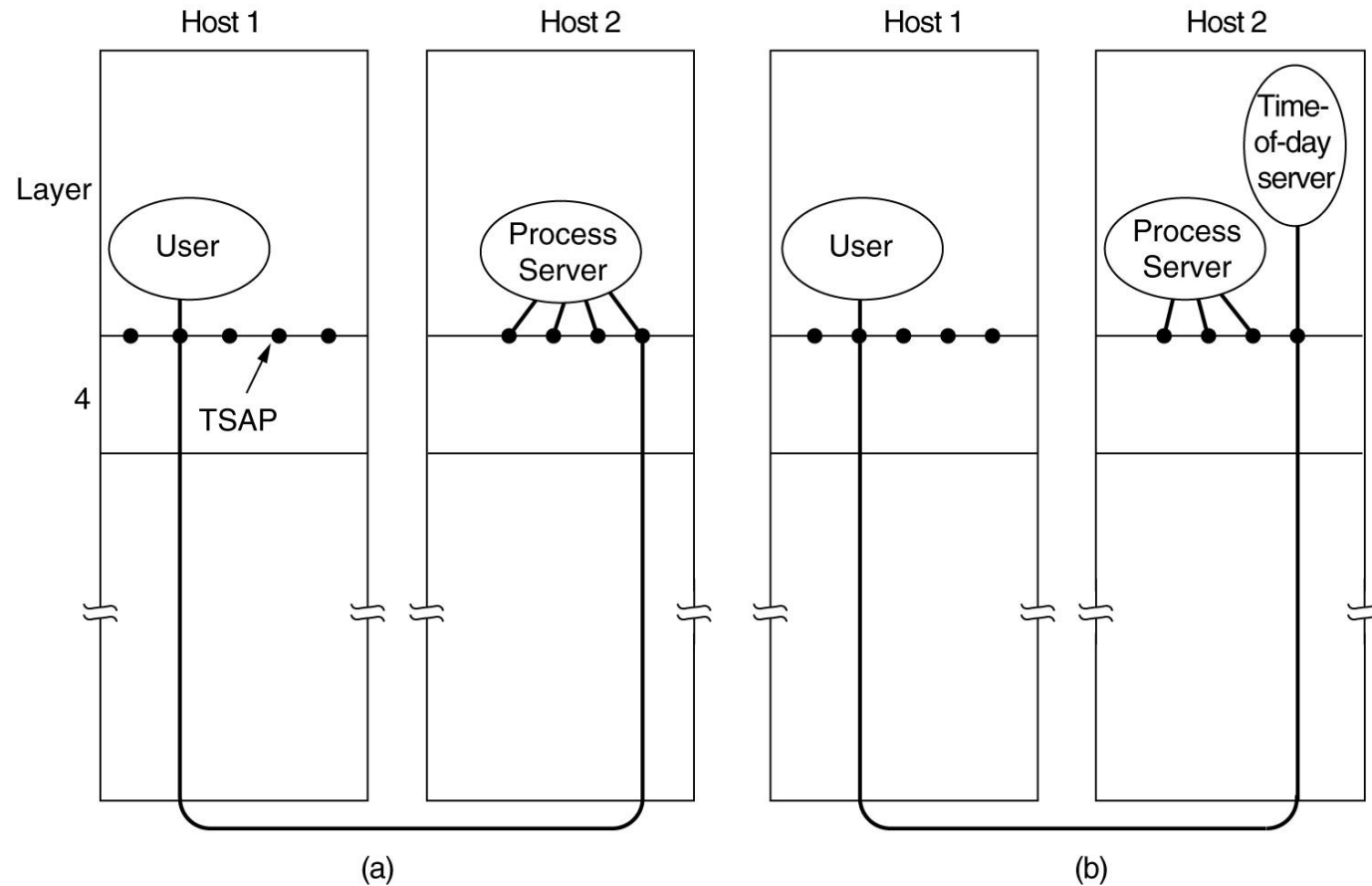
Figura 6a



1. **Escucha múltiple:** El servidor de procesos escucha en un grupo de puertos simultáneamente, esperando solicitudes de conexión entrantes de clientes.
2. **Solicitud CONNECT:** Un cliente emite una solicitud CONNECT, especificando el número de puerto del servicio que desea utilizar.

Protocolo Inicial de Conexión — Pasos 3 y 4

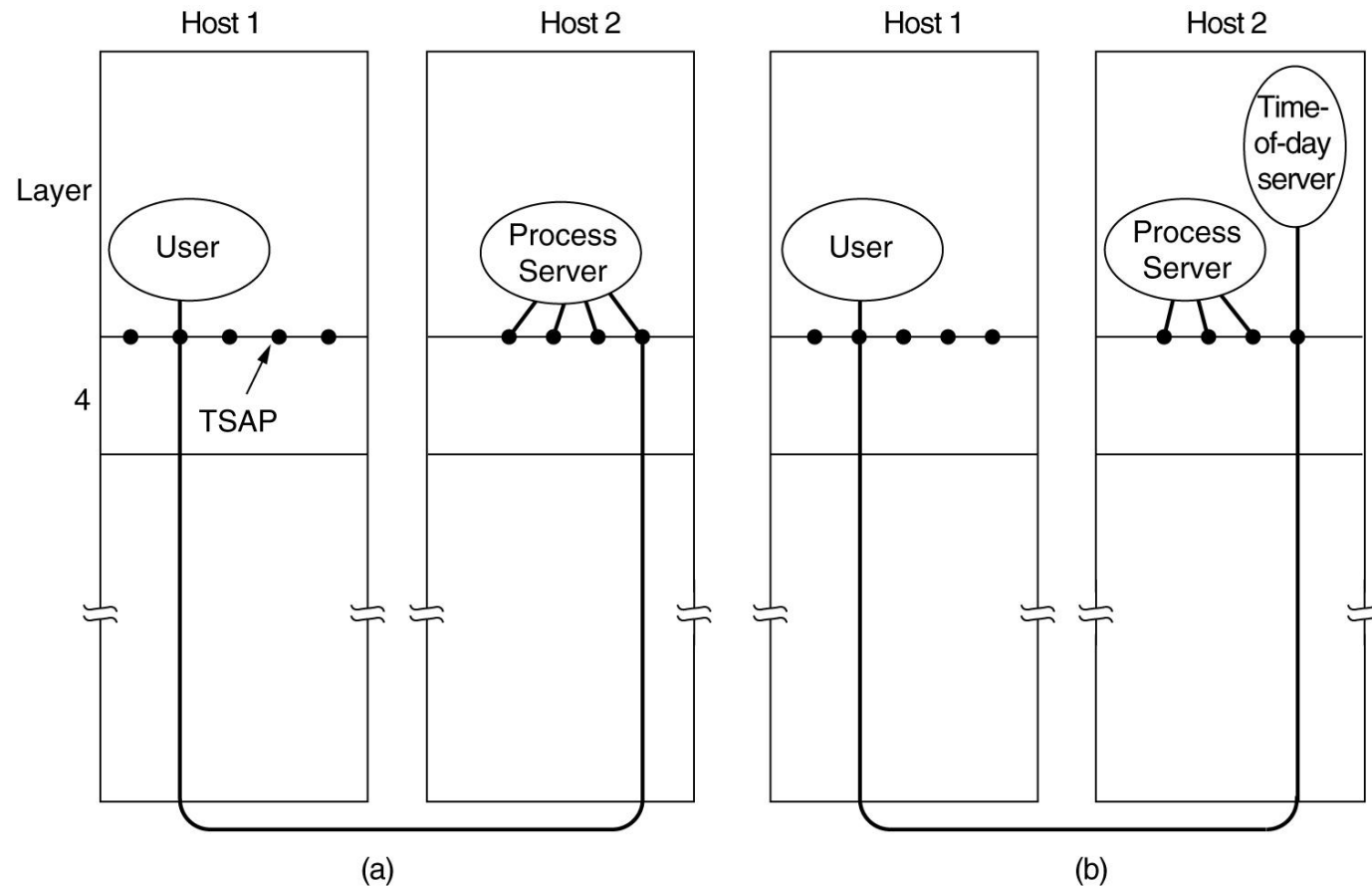
Figura 6b



- 3. Redirección:** Si no hay ningún servidor activo esperando en ese puerto, la solicitud llega al servidor de procesos, quien obtiene la conexión.
- 4. Generación (fork):** El servidor de procesos genera (fork) el servidor solicitado y le permite **heredar la conexión** existente con el cliente, de modo que la comunicación continúa sin interrupción.

Protocolo Inicial de Conexión — Paso 5 (Final)

Figura 6c



- Servicio activo:** El nuevo servidor recientemente activado realiza el trabajo solicitado por el cliente. Mientras tanto, el servidor de procesos vuelve a escuchar en los puertos, listo para atender nuevas solicitudes entrantes.

Ejercicio: Comparación de Soluciones

- **Ejercicio:** Responder las siguientes preguntas de análisis:
 - **Pregunta 1:** ¿Cuál es la diferencia fundamental entre la solución del *servidor de procesos* y la solución del *servidor de directorio*?
 - Pista: Pensar en qué problema resuelve cada uno. El directorio resuelve el *descubrimiento* del puerto; el servidor de procesos resuelve la *activación* del proceso.
 - **Respuesta:** El *servidor de directorio* resuelve el problema de **descubrimiento**: el cliente no sabe en qué puerto escucha el servicio, y el directorio le indica la dirección. Asume que el servidor ya está activo. El *servidor de procesos* resuelve el problema de **activación**: el cliente conoce el puerto, pero el servidor está inactivo, y el servidor de procesos lo genera (fork) bajo demanda y le transfiere la conexión.
 - **Pregunta 2:** ¿En qué escenario haría falta combinar ambas soluciones?
 - Pista: Considerar el caso en que el cliente no conoce al servidor **y además** el servidor está inactivo.
 - **Respuesta:** Cuando el cliente no conoce el puerto del servicio **y además** el servidor está inactivo. Primero se consulta al *servidor de directorio* para descubrir el puerto, y luego el *servidor de procesos* se encarga de activar el demonio correspondiente para atender la solicitud. Se combinan ambos mecanismos en secuencia: descubrimiento + activación.

Direccionamiento en TCP: Puertos Bien Conocidos

- **Puertos bien conocidos (well-known ports)**

- Son los puertos con números **menores a 1024**, reservados para servicios estándar del sistema. Estos números están asignados por la IANA (Internet Assigned Numbers Authority).

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing

- **Demonios (daemons)**

- Son procesos servidores que se ejecutan en segundo plano y se asocian a un puerto específico en el momento del arranque del sistema.
 - Ejemplo: El *demonio FTP* se conecta al **puerto 21** durante el arranque y queda escuchando solicitudes de transferencia de archivos.

El Demonio inetd — Solución al Consumo de Memoria

- **Problema:** Si cada servicio tiene su propio demonio ejecutándose permanentemente, se podría **llenar la memoria** con demonios que están inactivos la mayor parte del tiempo, desperdiciando recursos del sistema.
- **Solución:** Un único demonio llamado **inetd** (**Internet daemon**) reemplaza a múltiples demonios individuales. Este proceso:
 - Escucha simultáneamente en un **conjunto de puertos** y espera solicitudes de conexión entrantes.
 - Cuando un cliente emite un **pedido CONNECT** a un puerto específico, inetd verifica si ya hay un servidor escuchando en ese puerto.
 - Si **no hay servidor activo**, inetd **bifurca (fork)** un nuevo proceso, ejecuta el demonio apropiado en él, y ese demonio maneja la solicitud del cliente.

Configuración de inetd y Demonios Permanentes

- **Archivo de configuración de inetd**

- inetd determina qué puertos debe supervisar a partir de un **archivo de configuración** (generalmente */etc/inetd.conf*). Este archivo lista los servicios disponibles, sus puertos y los programas a ejecutar.
 - Los demonios asociados a estos puertos **solo se activan cuando hay trabajo**, ahorrando memoria y recursos del sistema.

- **Estrategia híbrida: Demonios permanentes + inetd**

- En la práctica se usa una **estrategia combinada**:
 - **Demonios permanentes** en los puertos más utilizados (ej: HTTP puerto 80, SSH puerto 22), porque el costo de tenerlos siempre activos se justifica por su alta demanda.
 - **inetd** gestiona los demás servicios de menor uso, activándolos bajo demanda para ahorrar recursos.
- Esta decisión la toma el **administrador del sistema**, según el perfil de uso de cada servidor.

Servidores de Directorios y Servicios en la Práctica

- **Servidor de Directorio — Ejemplos reales:**
 - **DNS (Domain Name System):** Traduce nombres de dominio a direcciones IP. Es el directorio distribuido más utilizado a escala global. [Capa de Aplicación]
 - **LDAP:** Usado en redes corporativas para localizar servicios y recursos. [Capa de Aplicación]
- **Servidor de Procesos (activación bajo demanda) — Ejemplos reales:**
 - **inetd / xinetd:** Implementación clásica en Unix/Linux. [Capa de Transporte – escucha en puertos TCP/UDP]
 - **systemd socket activation:** Reemplazo moderno de inetd en Linux. [Capa de Transporte – sockets TCP/UDP]
 - **AWS Lambda / Serverless:** Los procesos se generan bajo demanda. [Capa de Aplicación]
- **Combinación de ambos —** En **Kubernetes**, kube-dns (descubrimiento) + auto-scaling (activación). [Capa de Aplicación]

Nota: No todos corresponden a la capa de transporte. La capa de cada ejemplo se indica en [verde].

Ejercicio 1: Análisis de Escenarios

- Para cada situación, indicar qué mecanismo se necesita: **servidor de directorio**, **servidor de procesos**, **ambos** o **ninguno**. Justificar.
 - **a)** Un navegador web quiere conectarse al servidor HTTP de una máquina remota. Sabe que HTTP usa el puerto 80, y el servidor está activo permanentemente como demonio.
 - **b)** Una aplicación necesita un servicio de impresión remota. Sabe que el servicio existe en la máquina destino pero desconoce en qué puerto escucha.
 - **c)** Un cliente quiere usar el servicio finger (puerto 79) en un servidor Unix. El servicio está configurado en /etc/inetd.conf pero no hay ningún demonio finger corriendo actualmente.
 - **d)** Un proceso cliente necesita un servicio personalizado que fue recién instalado en el servidor. No conoce su puerto, y el servicio solo se activa bajo demanda mediante inetd.
 - **e)** Un administrador tiene un servidor con 50 servicios distintos pero solo 5 se usan frecuentemente. ¿Qué estrategia de direccionamiento/activación debería implementar para optimizar el uso de memoria?

Ejercicio 1: Respuestas

- **a) Ninguno.** El cliente ya conoce el puerto (80 es well-known) y el servidor está activo como demonio permanente. No se necesita descubrimiento ni activación.
- **b) Servidor de directorio.** El problema es de *descubrimiento*: el cliente no sabe el puerto. Consulta al directorio, que le devuelve la dirección del servicio de impresión. El servicio se asume activo.
- **c) Servidor de procesos (inetd).** El problema es de *activación*: el cliente conoce el puerto 79, pero el demonio está inactivo. inetd escucha en ese puerto, recibe la solicitud, genera (fork) el proceso finger y le transfiere la conexión.
- **d) Ambos.** Hay dos problemas simultáneos: no conoce el puerto (→ directorio) y el servicio se activa bajo demanda (→ servidor de procesos/inetd). Primero consulta al directorio para descubrir el puerto, luego inetd activa el servicio al recibir la conexión.
- **e) Estrategia híbrida.** Los 5 servicios frecuentes se configuran como **demonios permanentes** (evitar la latencia de fork). Los 45 restantes se gestionan con **inetd** para ahorrar memoria, activándolos solo bajo demanda.

Ejercicio 2: Verdadero o Falso

- Indicar si cada afirmación es **verdadera** o **falsa**. Si es falsa, corregirla.
 1. El servidor de directorio y el servidor de procesos resuelven el mismo problema de direccionamiento.
 2. Los puertos bien conocidos (well-known ports) son aquellos con números menores a 1024, reservados para servicios estándar.
 3. El demonio inetd necesita tener todos los servidores cargados en memoria permanentemente para poder redirigir las solicitudes.
 4. En el protocolo inicial de conexión, el servidor de procesos genera (fork) el servidor solicitado y le permite heredar la conexión existente con el cliente.
 5. Para registrar un nuevo servicio en un servidor de directorio, basta con que el cliente envíe una solicitud de conexión al puerto deseado.
 6. En la práctica, todos los servicios de un servidor Unix deberían ser gestionados por inetd para minimizar el consumo de memoria.
 7. El servidor de directorio necesita escuchar en un puerto bien conocido para que los clientes puedan contactarlo sin necesitar otro directorio.

Ejercicio 2: Respuestas

1. **FALSA.** Resuelven problemas diferentes: el directorio resuelve el *descubrimiento* del puerto; el servidor de procesos resuelve la *activación* del proceso servidor inactivo.
2. **VERDADERA.** Los puertos 0-1023 están reservados por IANA para servicios estándar (ej: HTTP=80, FTP=21, SSH=22).
3. **FALSA.** Justamente lo contrario: inetd evita tener los servidores en memoria. Solo inetd está activo, escuchando en múltiples puertos, y genera (fork) cada servidor bajo demanda cuando llega una solicitud.
4. **VERDADERA.** El servidor de procesos hace fork, creando un proceso hijo que hereda los descriptores de archivo (incluida la conexión TCP con el cliente), y el padre vuelve a escuchar.
5. **FALSA.** El registro lo hace el servidor (no el cliente): el nuevo servicio debe enviar su identificador y puerto al servidor de directorio para que este lo incluya en su tabla de búsqueda.
6. **FALSA.** La estrategia óptima es híbrida: los servicios de alto tráfico (HTTP, SSH) deben ser demonios permanentes para evitar la latencia de fork; solo los servicios esporádicos se gestionan con inetd.
7. **VERDADERA.** Si el propio directorio no tuviera un puerto conocido, se crearía un problema de recursividad infinita (¿quién me dice dónde está el directorio?). Por eso el directorio siempre usa un puerto well-known.

Ejercicio 3: Diseño de Solución

- **Situación:** Una universidad tiene un servidor central con 30 servicios disponibles para los alumnos (correo, FTP, SSH, bases de datos, impresoras, consulta de notas, etc.). Solo 4 servicios se usan constantemente (correo, SSH, DNS, HTTP). Los demás se usan esporádicamente.
- Se pide diseñar la estrategia de direccionamiento y activación del servidor, respondiendo:
 - a) **Clasificar:** ¿Cuáles servicios configuraría como demonios permanentes y cuáles gestionaría mediante inetd? Justificar el criterio utilizado.
 - b) **Directorio:** Un alumno quiere usar el servicio de “consulta de notas” pero no conoce su puerto. ¿Qué pasos seguiría el sistema para resolver esta solicitud?
 - c) **Nuevo servicio:** Si se agrega un servicio de “reserva de aulas”, ¿qué pasos debe realizar el administrador para que los clientes puedan descubrirlo y utilizarlo?
 - d) **Diagrama:** Dibujar el flujo completo de mensajes entre el cliente, el servidor de directorio e inetd cuando un alumno accede al servicio de “consulta de notas” por primera vez.

Ejercicio 3: Respuestas

- a) **Clasificar:** **Demonios permanentes:** Correo, SSH, DNS y HTTP (alto tráfico → evitar latencia de fork en cada solicitud). **Gestionados por inetd:** Los 26 restantes (FTP, bases de datos, impresoras, consulta de notas, etc.) → uso esporádico, no justifica ocupar memoria permanentemente. Criterio: frecuencia de uso y tiempo de respuesta aceptable.
- b) **Directorio:** 1) El cliente del alumno contacta al servidor de directorio (puerto well-known). 2) Envía un lookup con el identificador “consulta de notas”. 3) El directorio busca en su tabla y responde con el puerto asignado. 4) El cliente se conecta a ese puerto. 5) Como está gestionado por inetd, este recibe la conexión, hace fork del proceso y le transfiere la conexión.
- c) **Nuevo servicio:** El administrador debe: 1) Asignar un puerto al servicio “reserva de aulas”. 2) Agregar una entrada en /etc/inetd.conf con el puerto, protocolo y ruta del ejecutable. 3) Registrar el servicio en el servidor de directorio (enviar identificador + puerto). 4) Reiniciar inetd para que lea la nueva configuración. Los clientes podrán descubrirlo vía directorio y será activado bajo demanda por inetd.
- d) **Diagrama:** Cliente → Srv. Directorio: “¿puerto de consulta de notas?” → Directorio responde: “puerto N” → Cliente se conecta a puerto N → inetd (escuchando en N) acepta conexión → inetd hace fork del proceso “consulta_notas” → Hijo hereda conexión y atiende al cliente → inetd vuelve a escuchar.